Python 基础知识

目录

- 一、变量与赋值
- 1.1 变量
- 1.2 赋值
- 二、常用操作符
- 2.1 算数操作符
- 2.2 赋值操作符
- 2.3 比较操作符
- 2.4 逻辑操作符
- 三、数据类型
- 3.1 Number（数值）
- 3.2 String（字符串）
- 3.3 List（列表）
- 3.4 Dictionary（字典）
- 3.5 Tuple（元组）
- 3.6 Sets（集合）

一、变量与赋值

- 1.1变量
- 含义：是编程语言中能存储数据的抽象概念，它可以存储不同类型的数据。
- 隐喻：变量就像是水杯，能存放各种不同类型的饮料。

注意：
变量不能以数字开头；
变量不能包含空格；
变量大小写敏感(变量A和变量a不同)。

一、变量与赋值

- 1.2 赋值
- 含义:把一个数据放到变量中的过程就叫做赋值。赋值需要用到等号，所以等号又叫做“赋值运算符”
- 隐喻：赋值是把饮料倒入水杯的过程。

```
ABC = 1024
ThisYear = 2019
_a = 1
No122 = 30594484
```

二、常用操作符

- 2.1 算数操作符

操作符	实例
+ 加	3+2 >>> 5
- 减	3-2 >>> 1
* 乘	3*2 >>> 6
/ 除	3.0/2 >>> 1.5
% 取模	10%3 >>> 1
// 取商	10//3 >>> 3
** 指数	3**2 >>> 9

二、常用操作符

- 2.2 赋值操作符

操作符	实例
=	c = a + b
+=	c + = a相当于c = c + a
-=	c - = a相当于c = c - a
*=	c * = a 相当于c = c * a
/=	c / = a 相当于c = c / a

二、常用操作符

- 2.3 比较操作符

操作符	实例
==	3==2 >>> False
!=	3!=2 >>> True
<>	3<>2 >>> True （与!=效果相同）
>	3 > 2 >>> True
<	3 < 2 >>> False
>=	3 >= 2 >>> True
<=	3 <= 2 >>> False

二、常用操作符

- 2.4 逻辑操作符

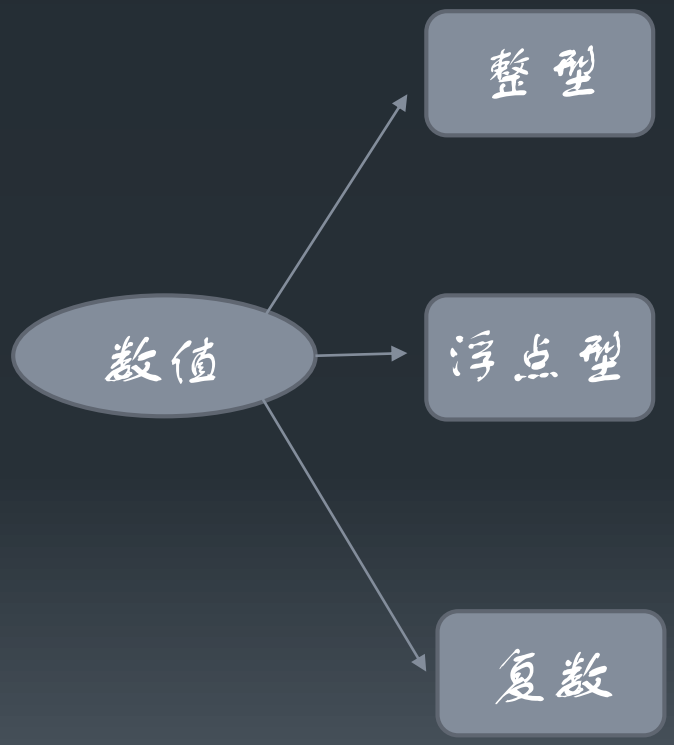
操作符	描述	实例
and	当且仅当两个操作数为真返回真，否则返回假	True and False >>> False
or	当且仅当有两个操作数至少一个为真则返回真，否则返回假	True or False >>> True
not	用于反转操作数的逻辑状态	not False >>> True

三、数据类型

- Python3 中有六个标准的数据类型：Number（数值）、String（字符串）、List（列表）、Tuple（元组）、Sets（集合）、Dictionary（字典）。
- 其中，按照数据是否可变：
- 可变数据：列表、字典、集合；
- 不可变数据：数值、字符串、元组。

三、数据类型

- 3.1 Number（数值）



整型(int) 通常被称为是整型或整数，是正或负整数，不带小数点。
整数运算结果仍是整数。（1+2>>>3)

浮点型(float) 由整数部分与小数部分组成，浮点类型不精确存储。
如果操作符两端其中一个操作数是浮点数，那么运算结果是浮点数。(1.0+2>>>3.0)

复数(complex)由实数部分和虚数部分构成，可以用a + bj,或者complex(a,b)表示， 复数的实部a和虚部b都是浮点型。

三、数据类型

- 3.1.1数字类型转换

- int(x) 将x转换为一个整数。
- float(x) 将x转换到一个浮点数。
- complex(x) 将x转换到一个复数，实数部分为 x，虚数部分为 0。

```
x=12.987
y=complex(x)
print(y)
```

→

```
(12.987+0j)
```

三、数据类型

- 3.2 String（字符串）

编程中表示文本的数据类型。创建字符串可以使用单引号、双引号、三单引号和三双引号，其中三引号可以多行定义字符串。

```
print("Hello World!")
print("0373")
print("Thank You!")
```

三、数据类型

- 3.2.1 字符串索引与切片
- 索引 str[index]

例：

正索引	0	1	2	3	4	5
值	p	y	t	h	o	n
负索引	-6	-5	-4	-3	-2	-1

```
x='python'
y=x[0]
print(y)
>>> p
```

三、数据类型

- 切片 str[start_index end_index step]

start_index—表示起点。

end_index—表示终点（但不包含该点）。

step —表述步数。取正数表示从左往右切片，取负数表示从右往左切片。当step省略时，默认step值为1。

- 切取完整对象

正索引	0	1	2	3	4	5
值	p	y	t	h	o	n
负索引	-6	-5	-4	-3	-2	-1

```
x='python'
y=x[:]
print(y)
>>> python
```

三、数据类型

- 切取特定对象 str[start_index end_index step]

正索引	0	1	2	3	4	5
值	p	y	t	h	o	n
负索引	-6	-5	-4	-3	-2	-1

```
x='python'
y=x[1:4:2]
print(y)
>>> yh
```

三、数据类型

- 3.2.2 字符串替换
- str.replace（old_str，new_str，[count]）

将字符串str中的字符old_str替换为新的new_str字符串， count是可选参数，不写的情况下，表示将str中所有的old_str替换为new_str ，写之后表示只有前面的count个出现被替换。

例：

```
x='python python'
y=x.replace('py','ab',1)
print(y)
>>> abthon python
```

三、数据类型

- 3.2.3 字符串大小写转换
- str.title()与str.capitalize()

str.title() 将字符串每个单词的首字母转为大写，其他字母转为小写；
str.capitalize() 将字符串第一个单词首字母转为大写。

例：

```
x='i love python'
y=x.title()
print(y)
>>> I Love Python
```

例：

```
x='i love python'
y=x.capitalize()
print(y)
>>> I love python
```

三、数据类型

- 3.2.3 字符串大小写转换
- str.lower()与str.upper()

str.lower() 方法用于将字符串中的所有大写字母转换为小写字母；str.upper()方法用于将字符串中的所有小写字母转换为大写字母。

例：

```
x='I Love Python'
y=x.lower()
print(y)
>>> i love python
```

例：

```
x='I Love Python'
y=x.upper()
print(y)
>>> I LOVE PYTHON
```

三、数据类型

- 3.2.5 字符串格式化输出

例1：print(f '{x}')

```
x='world'
y='python'
print(f'Hello {x} , I am {y}!')
>>> Hello world , I am python!
```

例2： print({ }.format( ))

```
print('Hello {} , I am {}!'.format('world','python'))
>>>Hello world , I am python!
```

三、数据类型

- 3.2.5 字符串格式化输出

例3：

```
t=22.3678
print('The temperature is {:.2f}C'.format(t))
```

>>>The temperature is 22.37C

数字	格式	输出	描述
3.1415926	{:.2f}	3.14	保留小数点后两位
3.1415926	{:+.2f}	3.14	带符号保留小数点后两位
-1	{:+.2f}	-1	带符号保留小数点后两位
2.71828	{:.0f}	3	不带小数
1000000	{:,}	1,000,000	以逗号分隔的数字格式
0.25	{:.2%}	25.00%	百分比格式
1000000000	{:.2e}	1.00E+09	指数记法
25	{0:b}	11001	转换成二进制
25	{0:d}	25	转换成十进制
25	{0:o}	31	转换成八进制

三、数据类型

- 3.2.6 字符串查找
- str.find(sub,start,end)

描述：返回字符串中找到的子字符串sub的最低索引，使得sub包含在切片str[start,end]中，如果未找到sub,则返回-1。

- sub —要索引的子字符串。
- start —索引的起始位置。默认值为0。
- end —索引的结束位置。默认值为字符串长度。

例：

```
x='Zootopia'
y=x.find('to')
print(y)
>>>3
```

三、数据类型

- 3.2.7 字符串拆分
- str.split(sep=None, maxsplit=-1) [n]

描述：拆分字符串。通过指定分隔符sep对字符串进行分割，并返回分割后的字符串列表。

- sep —分隔符，默认为空格,但不能为空即(")。
- maxsplit —最大分割参数，默认参数为-1。
- [n] —返回列表中下标为n的元素。列表索引的用法。

例：

```
x='Z o o t o p i a'
y=x.split()
print(y)
>>>['Z', 'o', 'o', 't', 'o', 'p', 'i', 'a']
```

三、数据类型

- 3.2.7 字符串连接
- sep.join(iterable)

描述：连接字符串数组。

- sep—分隔符。可以为空。
- iterable—要连接的变量 ，可以是字符串，元组，字典，列表等。

例：

```
x=['Z', 'o', 'o', 't', 'o', 'p', 'i', 'a']
y='_'.join(x)
print(y)
>>>Z_o_o_t_o_p_i_a
```

三、数据类型

- 3.2.7 统计字符串次数
- str.count(“char”, start,end)

描述：统计字符串里某个字符出现的次数。

- char —要统计的字符。
- star —索引字符串的起始位置，默认参数为0。
- end —索引字符串的结束位置，默认参数为字符串长度。

例：

```
x='abcqo-ab'
y=x.count('ab',1)
print(y)
>>>1
```

三、数据类型

- 3.2.7 字符串去除指定字符
- str.strip(chars)

描述：该函数的作用是去除字符串开头和结尾处指定的字符.

chars—要去除的字符，默认为空格、制表符或换行符。

- 换行符与制表符

名称	符号	描述
换行符	\n	让字符串换到下一行
制表符	\t	在开头添加制表符（默认四个空格）

例：

```
a = '\t111 aaa'
print(a)
b=a.strip()
print(b)
>>>    111 aaa
>>>111 aaa
```

```
#!/usr/bin/python

str = "     this is string example....wow!!!     ";
print str.lstrip();
str = "88888888this is string example....wow!!!8888888";
print str.lstrip('8');
```

三、数据类型

- 3.3 List（列表）

列表是写在方括号 [] 之间、用逗号分隔开的元素列表，列表可以完成大多数集合类的数据结构实现。列表中元素的类型可以不相同，它支持数字，字符串甚至可以包含列表（嵌套），列表中的元素是可以改变。

```
list1 = ['baidu', 'google', 12, 34];
list2 = [1, 2, 3, 4, 5 ];
list3 = ["a", "b", "c", "d"];
```

三、数据类型

- 3.3.1列表函数

函数名	描述
len()	返回列表中的元素个数
max()	返回列表元素最大值
min()	返回列表元素最小值

例：

```
list1 = ['baidu', 'google', 12, 23]
print(len(list1))
>>>4
```

三、数据类型

- 3.3.2列表方法

函数名称	描述
list.append(x)	添加一个x元素到列表末尾
list.extend(L)	表示追加内容，它可以在列表的末尾一次性追加另一个序列中的多个值，也就是用新列表扩展原有列表
list.insert(i,x)	将索引为i的元素位置前插入一个元素x
list.remove(x)	删除列表第一个值为x的元素
del list[i]	删除列表中索引值为i的元素
list.pop([i])	删除列表指定位置的元素并返回它，默认删除最后并返回列表的最后一个元素
list.index(x)	返回列表第一个值为x的元素下标
list.reverse()	反转列表中的元素
list.count(x)	返回列表出现x的次数
list.sort()	排序列表中的元素
list.clear()	清空列表

三、数据类型

- 3.3.2列表方法
- list.append(x)

例：

```
list1 = [1,2,3,4]
list1.append(5)
print(list1)
>>>[1, 2, 3, 4, 5]
```

三、数据类型

- 3.3.2列表方法
- list.extend(L)

例：

```
a =[1,2,3]
b = [4,5,6]
a.extend(b)
print(a)
>>>[1, 2, 3, 4, 5, 6]
```

注意：此操作类似于列表的相加操作，但是追加操作改变原有列表，而相加不改变原有列表，例如：

```
a =[1,2,3]
b = [4,5,6]
print(a+b)
print(a)
>>>[1, 2, 3, 4, 5, 6]
>>>[1,2,3]
```

三、数据类型

- 3.3.2列表方法
- list.insert(i,x)

例：

```
list2=[1,2,3,4]
list2.insert(1,5)
print(list2)
>>>[1, 5, 2, 3, 4]
```

三、数据类型

- 3.3.2列表方法
- list.remove(x)——知道具体元素值，不知道元素索引值

例：

```
content=['where', 'who', 'list', 'cntent', 'how', 'who']
content.remove('who')
print(content)
>>>['where', 'list', 'cntent', 'how', 'who’]
```

- del list[i]——不知道具体元素值，知道元素索引值

例：

```
content=['where', 'who', 'list', 'cntent', 'how', 'who’]
del content[1]
print(content)
>>>['where', 'list', 'cntent', 'how', 'who’]
```

三、数据类型

- 3.3.2列表方法
- list.pop([i])

例：

```
content=['where', 'who', 'list', 'cntent', 'how', 'who']
print(content.pop())
print(content)
>>>who
>>>['where', 'who', 'list', 'cntent', 'how']
```

三、数据类型

- 3.3.2列表方法
- list. index(x)

例：

```
content=['where', 'who', 'when', 'cntent', 'how', 'who']
print(content.index('when'))
>>>2
```

三、数据类型

- 3.3.2列表方法
- list.count(x)

例：

```
num=[1, 2, 3, 4, 5, 5, 5, 5, 6]
print(num.count(5))
>>>4
```

三、数据类型

- 3.3.2列表方法
- list. sort()

例：

```
x = [2,3,5,6,1,4,7]
x.sort()
print(x)
>>>[1, 2, 3, 4, 5, 6, 7]
```

sort 与 sorted 区别：

sort 是应用在 list 上的方法，sorted 可以对所有可迭代的对象进行排序操作。

list 的 sort 方法返回的是对已经存在的列表进行操作，无返回值，而内建函数 sorted 方法返回的是一个新的 list，而不是在原来的基础上进行的操作。

三、数据类型

- 3.3.2列表方法
- list. reverse()

例：

```
x=['BMW','TUYOUTA','Audi','Ford']
x.reverse()
print(x)
>>>['Ford', 'Audi', 'TUYOUTA', 'BMW']
```

三、数据类型

- 3.3.2列表方法
- list.clear()

例：

```
list3=['baidu', 'google', 12, 23]
list3.clear()
print(list3)
>>>[]
```

三、数据类型

- 3.3.3列表的基本操作

列表可以使用所有适用于序列（字符串、列表、元组、集合和字典）的标准操作，比如索引、切片、连接和相乘。重要的是，列表是可以修改的，也就是定义的列表内容可以根据需求更改，如元素赋值、元素删除等。

- 元素赋值

例：

```
x=[1,2,3,4,5]
x[2]=8
print(x)
>>>[1, 2, 8, 4, 5]
```

三、数据类型

- 元素删除

例：

```
names=['zhangsan','lisi','wangwu','zhaoliu']
del names[2]
print(names)
>>>['zhangsan', 'lisi', 'zhaoliu']
```

三、数据类型

3.4 Dictionary（字典）

Python 中的字典提供了一种灵活的访问和组织数据的方式:

- 字典是由很多值组成的集合;
- 字典的索引可以是不同的数据类型，不止是整数，也有字符串;
- 字典的索引被称为“键”，键及键所关联的值叫键值对;
- 字典是一种可变容器模型，且可存储任意类型对象。
- 字典的每个键(key)与值(value) 用冒号 分割，每个键值对之间用逗号 分割，整个字典包括在花括号 {} 中 ,格式如下所示:

```
Price = {"banana":5,"tissue":1,"milk":3}
print(Price)
print(Price["tissue"])
```

三、数据类型

- 3.4.1访问字典的数据
- D[key]

例：

```
MyDog = {'size':'big','color':'white','character':'gentle'}
print(MyDog['size'])
>>>big
```

三、数据类型

- 3.4.2添加字典键值对
- D[key]=value

例：

- MyDog={'size':'big','color':'white'}
- MyDog['food']='beef'
- print(MyDog)

```
>>>{'size': 'big', 'color': 'white', 'food': 'beef'}
```

三、数据类型

- 3.4.3修改字典元素
- D[key]=new_value

例：

```
MyDog = {'size':'big','color':'white','character':'gentle'}
MyDog['size']='samll'
print(MyDog)
>>>{'size': 'samll', 'color': 'white', 'character':
'gentle'}
```

三、数据类型

- 3.4.4字典键的特性:
- 不允许同一个键出现两次。创建时如果同一个键被赋值两次，后一个值会被记住;

例:

```
dict = {'Name': 'Fiona', 'Age': 10, 'Name': 'Manni’}
print ( dict['Name'])
>>>Manni
```

- 键必须不可变(键可以用数字，字符串或元组充当)，值可变（可以用列表充当）。

例:

```
dict = {'Name': ['Fiona', 'Bob'],'Age': 10}
print (dict['Name'])
>>> ['Fiona', 'Bob']
```

三、数据类型

- 3.4.5字典的方法
- dict.clear()—删除字典内所有元素

例：

```
dict = {'Name': 'Fiona', 'Age': 10}
print (f'字典长度:{len(dict)}')
dict.clear()
print (f'字典删除后长度:{len(dict)}')
>>>字典长度:2
>>>字典删除后长度:0
```

三、数据类型

- 3.4.5字典的方法
- dict.copy()—对字典进行复制(深拷贝)
- 深拷贝不会随原始的dict修改而修改。

例:

```
dict1={'user':'runoob'}
dict2=dict1.copy() #深拷贝
dict1['user']='root'#修改数据
print(dict1)
print(dict2)
>>>{'user': 'root'}
>>>{'user': 'runoob'}
```

三、数据类型

- 3.4.5字典的方法
- dict_origin=dict_new—对字典进行引用(浅拷贝）
- 浅拷贝会随 原始的dict 的修改而修改。

例：

```
dict1={'user':'runoob'}
dict3=dict1#浅拷贝
dict1['user']='root'#修改数据
print(dict1)
print(dict3)
>>>{'user': 'root'}
>>>{'user': 'root'}
```

三、数据类型

- 3.4.5字典的方法
- dict.get(key)—返回指定键的值，如果值不在字典中返回default值。

例:

```
dict = {'Name': 'Mary', 'Age': 20}
Myfriend_Age=dict.get('Age')
print(f'My friend’s  age is {My_Age}')
>>>My friend’s age is 20
```

三、数据类型

3.5 Tuple（元组）

元组（tuple）与列表类似（戴了枷锁的列表），不同之处在于元组的元素不能修改。元组写在小括号（）里，元素之间用逗号隔开，组中的元素类型也可以不相同。

```
nametuple = ("sandy","sue","billy")
numbertuple = (3,10,237)
print(nametuple[1])
```

```
>>> TupNum = (34)
>>> type(TupNum)    # 不加逗号是整型
<class 'int'>
>>> TupNum = (34,)
>>> type(TupNum)   # 加上逗号变元组
<class 'tuple'>
```

注意：元组中只包含一个元素时，需要在元素后面添加逗号，否则括号会被当作运算符使用。

三、数据类型

- 元组的必要性：

既然有列表的存在，为什么还需要元组那样不可变序列呢？

元组的不可变性从某种角度说，是它的天然优势。例如，字典里的键值必须保证是唯一的，不可修改的。所以，允许元组作为键值，但不允许列表作为键值。

三、数据类型

- 3.5 Tuple（元组）
- 3.5.1 访问元组
- 元组的访问和序列访问元素一样，都是通过下标索引进行访问操作

例：

```
tup1 = ('baidu', 'google',1,2)
print(tup1[0:2])
```

>>>('baidu', 'google')

三、数据类型

- 3.5.2 修改元组
- 元组中的值一旦定义就不能修改，但是我们可以通过元组与元组之间的连接关系来对元组进行修改。或者重新定义元组进行修改。

例1:

```
tup1 = ('baidu', 'google',1,2)
tup2 = (1, 2, 3, 4, 5, 6, 7)
print(tup1 + tup2)
>>>('baidu', 'google', 1, 2, 1, 2, 3, 4, 5, 6, 7)
```

例2:

```
tup1 = ('baidu', 'google',1,2)
tup1 = (1, 2, 3, 4, 5, 6, 7)
print(tup1)
>>>(1, 2, 3, 4, 5, 6, 7)
```

三、数据类型

- 3.5.3 删除元组
- 由于元组的不可修改性，所以元组中的元素值是不允许删除的，但我们可以使用 del 语句来删除整个元组。

例：

```
tup1 = ('baidu', 'google',1,2)
print (tup1)
del tup1
print ("删除后的元组 tup : ")
>>>删除后的元组 tup :
```

三、数据类型

- 3.5.4 元组运算符
- 元组求长度——函数 len()

例：

```
tup1 = ('baidu', 'google',1,2)
print(len(tup1))
>>>4
```

- 连接元组——使用 + 连接符

方法同修改元组

- 复制元组——使用*复制

例：

```
tup1 = ('abc')
print((tup1,) * 3)
>>>('abc', 'abc', 'abc')
```

三、数据类型

- 3.5.5 元组内置函数

函数名	描述
len(tuple)	计算元组元素个数
max(tuple)	返回元组收纳柜元素最大值
min(tuple)	返回元组收纳柜元素最小值
tuple(list)	将列表转换为元组

例：

```
list1 = ['abc',123,'mn']
print(tuple(list1))
>>>('abc', 123, 'mn')
```

三、数据类型

- 3.6 Sets（集合）

集合是由不重复元素组成的无序的集。它的基本用法包括成员检测和消除重复元素。集合对象也支持像并集，交集，差集，对称差分等数学运算。集合就像特殊的字典，相当于字典中的key。

```
set1 = {'hello', 'hello', 'word', 'word'}
set1
#  输出结果实现自动去重
{'hello', 'word'}
```

三、数据类型

- 3.6.1 添加元素
- s.add(x)

例:

```
s = set(('hello','world'))#创建集合
print(s)
s.add('!')#添加集合元素
print(s)
>>>{'hello', 'world’}
>>>{'!', 'hello', 'world'}
```

三、数据类型

- 3.6.2 移除元素
- s.remove(x)

例：

```
s = set(('hello','world'))
s.remove('world')
print(s)
>>>{'hello’}
```

注意：集合不支持索引和切片，所以不能使用下标索引移除集合元素

例：

```
s = set(('hello','world'))
del s[1]
print(s)
>>>
```

```
Traceback (most recent call last):
  File "e:/pyrhon code/yequ/basic/practice.py", line 2, in <module>
    del s[1]
TypeError: 'set' object doesn't support item deletion
```

三、数据类型

- 3.6.3 计算元素个数
- len(s)

例：

```
s = set(('hello','world'))
print(len(s))
>>>2
```

- 3.6.4 清空集合元素
- s.clear()

例：

```
s = set(('hello','world'))
s.clear()
print(s)
>>>set()
```

三、数据类型

- 3.6.5 集合运算
- 集合之间的运算符分别是‘-’、‘|’、‘&’、‘^’
- ‘-’：代表前者中包含后者中不包含的元素
- ‘|’：代表两者中全部元素聚在一起去重后的结果
- ‘&’：两者中都包含的元素
- ‘^’：不同时包含于两个集合中的元素

三、数据类型

- 3.6.5 集合运算

例：

```
S1=set((1,4,6,8,9))
S2=set((2,4,5,6,8))
print(S1-S2)#S1集合中有，而S2集合没有的元素
print(S1|S2)#S1集合、S2集合包含的所有元素（去重复后）
print(S1&S2)#S1集合和S2集合中都有的元素
print(S1^S2)#不同时包含于S1集合、S2集合的元素
>>>{1, 9}
>>>{1, 2, 4, 5, 6, 8, 9}
>>>{8, 4, 6}
>>>{1, 2, 5, 9}
```

三、数据类型

- 3.6.6 集合内置函数
- set.difference(set)
- 用于返回集合的差集，即返回的集合元素包含在第一个集合中，但不包含在第二个集合(方法的参数)中，返回一个新的集合。

例：

```
# 求两个集合的差集，元素在 x 中不在 y 中
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
z = x.difference(y)
print(f'两个集合的差集是:{z}')
```

```
>>>两个集合的差集是: {'banana', 'cherry'}
```